

شبکه

CA1

سید علیرضا میرشفیعی

810101532

مقدمه:

پروژه در رابطه با یک نسخه ساده از پروتکل FTP در قالب یک چت گروهی. در ادامه هریک از بخش های Client و Server را همراه با ساختار کلی و نحوه اجرا و ... توضیح میدهیم.

:Client

برنامه client صرفا قرار است که به سرور با استفاده از TCP به سرور متصل شود و بتواند هر پیامی را به سمت سرور بفرستد، و تا جای ممکن پردازش این پیام ها از طرف سرور اتفاق بیفتد و client بتواند آزادانه هر پیامی را ارسال کند.

معماری:

ساختار کلاس client حاوی اطلاعات مربوط به سوکت، پورت، ip و استراکت مربوط به sockaddr میباشد که برای بخش اتصال آن به سرور مورد نیاز هستند. همچنین حاوی یک آدرس برای فولدر ذخیره سازی داده میباشد. از آنجایی که برای موازی سازی در هر دو برنامه client و سرور از thread استفاده شده است، در برنامه از قفل mutex یا متغیرهای atomic نیز استفاده کرده ایم تا از هرگونه race condition جلوگیری شود.

```
class Client {
private:
    int client_socket;
    int server_port;
    std::string server_ip;
    struct sockaddr_in server_addr;

    std::atomic<bool> is_uploading{false};
    std::atomic<bool> is_downloading{false};
    std::atomic<long> expected_filesize{0};
    std::string current_filename;

    const std::string client_storage_path = "client_storage/";
};
```

:Connection

برای اتصال کلاینت به سرور، اول یک سوکت به عنوان نماینده کلاینت `setup` میشود که در آن `AF_INET` به معنی دامنه `ip` هاست که همان دامنه `ipv4` میباشد. و `SOCK_STREAM` همان `type` مورد استفاده برای پروتکل `TCP` میباشد. (اگر میخواستیم از `UDP` استفاده کنیم `SOCK_DGRAM` استفاده میکردیم) از `inet_pton` نیز برای تبدیل آیپی متنی به آیپی باینری در ساختار شبکه استفاده میشود.

```
void Client::setupSocket() {
    client_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (client_socket == -1) {
        std::cerr << "Error: Could not create socket!" << std::endl;
        exit(EXIT_FAILURE);
    }

    // تبدیل آیپی متنی به آیپی باینری در ساختار شبکه
    if (inet_pton(AF_INET, server_ip.c_str(), &server_addr.sin_addr) <= 0) {
        std::cerr << "Error: Invalid address / Address not supported!" << std::endl;
        exit(EXIT_FAILURE);
    }
}
```

برای اتصال به سرور نیز از `connect` استفاده میشود.

```
void Client::connectToServer() {
    if (connect(client_socket, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        std::cerr << "Error: Connection to server failed!" << std::endl;
        exit(EXIT_FAILURE);
    }
    std::cout << "[+] Connected to the server successfully!" << std::endl;
}
```

تنها بخشی که در سمت کلاینت دارای کمی پیش پردازش بخصوص و با قاعده خواهد بود مربوط به دستورات `PUT` و `GET` میباشد که نیاز است به شکل قرار دادی میان سرور و کلاینت باشد. که در این معماری بصورت `handshake` این قرار داد صورت گرفته است.

`PUT`: اول کلاینت یک درخواست ارسال میکند که میخواد فایلی را ارسال کند، سپس سرور یک پیام به عنوان سیگنال `READY` میفرستد که یعنی آماده دریافت فایل میباشد، سپس فایل از سمت کلاینت ارسال میشود.

`GET`: درخواست مربوط به فایل از طرف کلاینت به سرور ارسال میشود، سپس منتظر سیگنال ارسال از طرف سرور میمانده، پس از گرفتن سیگنال فایل از طرف سرور به کلاینت ارسال میشود.

بقیه دستورات کاملاً از طرف سرور هندل خواهند شد و کلاینت صرفاً هر پیامی که بخواهد را میتواند برای سرور ارسال کند.

:Server

معماری:

همه پردازش ها در این برنامه اتفاق خواهد افتاد، که حاوی دو class خواهد بود، یکی خود server که بخش connection ها با کلاینت هارا مدیریت کند، و یک class دیگر به عنوان client_handler تا پیام هایی که کلاینت ها به سمت سرور ارسال میکنند را مدیریت کند و پاسخ دهد.

مانند قبل از آنجایی که از thread استفاده میشود برای جلوگیری از race condition از قفل mutex استفاده میکنیم.

```
class Server {
private:
    int server_fd;
    int port;
    struct sockaddr_in server_addr;

    std::map<int, ClientInfo> clients;
    std::mutex clients_mutex;

    ClientHandler client_handler;
```

:Connection

یک سوکت ساخته میشود که به عنوان نماینده سرور خواهد بود همچنین استفاده از تابع setsockopt با پارامتر SO_REUSEADDR باعث میشود که اگر سرور متوقف شد و بلافاصله دوباره اجرا گردید، بدون دریافت خطای مسدود بودن پورت، بتواند مجدداً از همان پورت قبلی استفاده کند.

```
void Server::setupSocket() {
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        std::cerr << "Error: Could not create socket!" << std::endl;
        exit(EXIT_FAILURE);
    }

    int opt = 1;
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
        std::cerr << "Error: setsockopt failed!" << std::endl;
        exit(EXIT_FAILURE);
    }
}
```

در ادامه با استفاده از تابع bind سوکت سرور را به یک آدرس و پورت مشخص متصل میکنیم. و پس از آن سرور با استفاده از تابع listen سوکتش را به یک حالت passive تبدیل میکند تا با داشتن یک صف ورودی بتواند منتظر اتصالات ورودی به آن بماند.

```

void Server::bindSocket() {
    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        std::cerr << "Error: Bind failed on port " << port << std::endl;
        exit(EXIT_FAILURE);
    }
}

void Server::startListening(int Client_limit) {
    if (listen(server_fd, Client_limit) < 0) {
        std::cerr << "Error: Listen failed!" << std::endl;
        exit(EXIT_FAILURE);
    }
    std::cout << "[+] Server is listening on port " << port << "..." << std::endl;
}

```

در قدم بعد تابع `accept` به صفی که توسط `listen` ساخته شده است نگاه میکند، اگر کلاینتی درون این صف منتظر اتصال بود، آن راه `pop` میکند و به آن متصل میشود و یک سوکت جدید به عنوان سوکت اختصاصی مربوط به این کلاینت میسازد و آن را برمیگرداند. در ادامه یک `thread` ساخته میشود تا تابع `handleClient` را که مربوط به فراخوانی `client_handler class` میباشد اجرا کند و مسئول رسیدگی به آن کلاینت بر روی سوکت جدید باشد. ساختار کلاس `ClientHandler` نیز بسیار ساده است و صرفاً قرار است که دستوراتی که از طرف کلاینت به سرور ارسال شده را هندل کند.

:Run

برای اجرای برنامه از `tmux` درون `makefile` استفاده شده تا روند تست برنامه و اجرای راحت تر باشد. در `tmux` 4 پنجره (ترمینال) ساخته میشود و در یکی از آن ها برنامه سرور و در 3 پنجره دیگر برنامه کلاینت اجرا میشود. در ادامه

:HELP

```

=== Available Commands ===
USERS                : Show a list of all connected users.
MSG <message>        : Send a broadcast message to all users.
PM <user_id> <message>: Send a private message to a specific user.
LIST                 : Show all files available on the server.
PUT <filename>        : Upload a file to the server.
GET <filename>        : Download a file from the server.
QUIT                 : Disconnect from the server.
HELP                 : Show this help message.
=====
> █

```

:USERS

```
cd Server && ./server || echo "Server failed"
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Server && ./server || echo "Server failed"
[+] Server is listening on port 8080...
[+] New client connected with FD: 4
[+] New client connected with FD: 5
[+] New client connected with FD: 6

alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
> USERS
--- Online Users ---
- User_4 (FD: 4)
- User_5 (FD: 5)
- User_6 (FD: 6)
-----
> █

cd Client && ./client
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
>

cd Client && ./client
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
>
```

:MSG

```
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
> USERS
>
--- Online Users ---
- User_4 (FD: 4)
- User_5 (FD: 5)
- User_6 (FD: 6)
-----
> MSG hello everyone
> █

cd Client && ./client
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
>
[Public from: User_4]: hello everyone
>

cd Client && ./client
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
>
[Public from: User_4]: hello everyone
>
```

:PM

```
> USERS
>
--- Online Users ---
- User_4 (FD: 4)
- User_5 (FD: 5)
- User_6 (FD: 6)
-----

> MSG hello everyone
> PM User_6 chetori gol pesar
>
Server: Private message sent to User_6
> █

cd Client && ./client
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
>
[Public from: User_4]: hello everyone
>

cd Client && ./client
alireza@alireza-VMware-Virtual-Platform:~/Documents/GitHub/Network_CAs/CA1$ cd Client && ./client
[+] Connected to the server successfully!
>
[Public from: User_4]: hello everyone
>
(Private from User_4): chetori gol pesar
>

"alireza-VMware-Virtua" 23:15 15-Apr-26
```

:LIST

```
- User_6 (FD: 6)
-----

> MSG hello everyone
> PM User_6 chetori gol pesar
>
Server: Private message sent to User_6
> LIST
>
--- Files on Server ---
- test3.txt (29 bytes)
-----
> █
```

:PUT

```
> PUT text1.txt
[Local Error]: File 'text1.txt' not found in client_storage/.
> PUT test1.txt
[System]: File uploaded.
>
Server: File 'test1.txt' uploaded successfully.
> LIST
>
--- Files on Server ---
- test3.txt (29 bytes)
- test1.txt (18 bytes)
-----
> █
```

:GET

```
> MSG hello everyone
> PM User_6 chetori gol pesar
>
Server: Private message sent to User_6
> LIST
>
--- Files on Server ---
- test3.txt (29 bytes)
-----
> GET test3.txt
>
[System]: File 'test3.txt' downloaded successfully.
> █
```

```
▼ client_storage ●
  ≡ test1.txt
  ≡ test2.txt
  ≡ test3.txt U
```